

SOMALI INTERNATIONAL UNIVERSITY • CS-501

# Week 2 — Your Professional Toolkit

VS Code • Terminal • Browser DevTools • Git and GitHub

Phase A | No AI for graded work | Semester Five

# Before we start: last week's checkpoint

## You should be able to answer

- What happens when you press Enter on a URL?
- Who does the work: the browser or the server?
- What does a 404 actually mean?
- What does HTTPS protect, and what does it not?

## Today we build the tools

Last week you learned how the Web works. This week you set up the environment you will use for the next 31 weeks — and you start keeping a permanent, graded record of your work.

## From today: if it is not in Git, it does not exist.

Every lab, exercise and project in this course is graded from your GitHub repository. Work saved only on your laptop earns zero marks.

# What we cover today

1

## VS Code

Configure the editor you will live in.

2

## The terminal

Move, create and inspect files without a mouse.

3

## Browser DevTools

See what the browser really does.

4

## Version control

Why Git exists and what problem it solves.

5

## Git basics

init, status, add, commit, log, .gitignore

6

## GitHub

Remote repositories, push, clone, commit messages.

PART ONE

# The Editor: Visual Studio Code

*A carpenter sharpens the saw before cutting. Spend twenty minutes now to save a hundred hours later.*

# Set up VS Code properly, once

## Extensions to install

- Prettier — automatic formatting
- ESLint — catches mistakes as you type
- Live Server — instant browser preview
- GitLens — see who changed what
- Error Lens — errors shown inline

## Settings to turn on

- Format on save
- Auto save after delay
- Word wrap
- Render whitespace
- Tab size = 2 for web projects

**One workspace folder per project. Never open loose files — open the folder. VS Code's terminal, Git panel and search all work on the open folder.**

# Shortcuts worth memorising this week

**Ctrl + `**

Open the integrated terminal

**Ctrl + P**

Jump to any file by name

**Ctrl + Shift + P**

Command palette — everything lives here

**Alt + ↑ / ↓**

Move the current line up or down

**Shift + Alt + ↓**

Duplicate the current line

**Ctrl + D**

Select the next occurrence of the word

**Ctrl + /**

Comment or uncomment the line

**Ctrl + B**

Show or hide the sidebar

*On macOS, replace Ctrl with Cmd.*

## PART TWO

# The Terminal

*The window where professional tools actually live. Every tool in this course — npm, git, node, psql — is a command.*

# The mental model: you are standing in a folder

## Three questions, always

**Where am I?** `pwd`

**What is here?** `ls` (dir on Windows CMD)

**How do I move?** `cd folder-name`

*The terminal never guesses. It does exactly what you type, in the folder you are standing in.*

```
$ pwd
/home/student/projects

$ ls
week01/  week02/  notes.txt

$ cd week02
$ pwd
/home/student/projects/week02
```

**Absolute path starts from the root: /home/student/projects. Relative path starts from where you are: ../week01**



# The commands you will use every day

**pwd**

Print the folder you are in

**ls**

List what is in this folder

**cd name**

Enter a folder

**cd ..**

Go up one level

**mkdir name**

Create a folder

**touch file.txt**

Create an empty file

**cp a b**

Copy a file

**mv a b**

Move or rename

**rm file**

Delete a file — no recycle bin

**cat file**

Print a file to the screen

**code .**

Open this folder in VS Code

**clear**

Clean the screen

**Careful: rm deletes permanently. There is no recycle bin in the terminal.**

PART THREE

# Browser DevTools

*Your window into what the browser is actually doing — not what you hoped it was doing.*

# Five panels, five jobs

## Elements

Inspect and edit the live HTML and CSS. Changes here are temporary — they disappear on reload.

## Console

Read errors and run JavaScript against the live page. Your first stop when something breaks.

## Network

Every request: method, status code, size and time. Your first stop when data does not arrive.

## Sources

The real debugger. Set breakpoints and step through code line by line — we use this from Week 6.

## Application

Local Storage, session storage and cookies. We use this from Week 8.

## PART FOUR

# Version Control: Why Git Exists

*Every professional team on earth uses version control. Not because it is fashionable — because the alternative is chaos.*

# The problem Git solves

## Without version control

```
project-final.zip  
project-final2.zip  
project-final-REAL.zip  
project-final-REAL-fixed.zip
```

- No record of who changed what, or why
- One overwritten file loses a week of work
- Two people cannot work at the same time

## With Git

- A full, dated history of every change
- Every change is attached to a person and a reason
- Go back to any previous state in seconds
- Work on a new feature without breaking the working version
- Several people can work on one project safely

**Git is a time machine with an explanation attached to every stop.**

# Git's three areas — the whole mental model

## Working directory

The files you are editing right now

`your folder`



## Staging area

The changes you have chosen to include in the next snapshot

`git add`



## Repository

The permanent, dated history

`git commit`

**A commit is a snapshot of the staged changes, with your name, the date, and your explanation attached. Nothing is committed until you stage it first.**

# The daily Git cycle

```
# once per project
git init

# every working session
git status
git add index.html
git add .
git commit -m "feat: add contact form"
git log --oneline

# send it to GitHub
git push
```

## `git status`

Your most-used command. Run it constantly.

## `git add`

Choose what goes into the next snapshot.

## `git commit`

Save the snapshot with a message.

## `git log`

Read the history you created.

## `git push`

Upload your history to GitHub.

*When in doubt: `git status`. It tells you exactly where you are and what to do next.*

# What must never enter your repository

## Create a .gitignore file

Git ignores every file or folder listed in it. Create it before your first commit.

```
node_modules/  
.env  
.DS_Store  
dist/
```

## Secrets

Passwords, API keys and .env files. A secret pushed to GitHub is a leaked secret, even if you delete it later.

## Huge folders

node\_modules can hold 200 MB of files anyone can reinstall in seconds.

## Machine noise

Editor settings and operating system files that mean nothing to your teammates.

In Semester 6 a leaked secret in a team repository costs marks in the security review. Start the habit now.



# From your laptop to GitHub

1

## Create the repository

On GitHub, empty – no README yet.

2

## Connect it

```
git remote add origin <url>
```

3

## Name the branch

```
git branch -M main
```

4

## Push the first time

```
git push -u origin main
```

5

## After that

```
git push – nothing more to type.
```

## Git vs GitHub

Git is the tool that runs on your computer and records history.

GitHub is a website that stores a copy of that history so you can share it, back it up, and collaborate.

You can use Git with no internet. You need GitHub to submit your work.

# Write commit messages a stranger can read

## Weak

```
update  
fix  
final  
asdf  
changes to the file
```

## Professional

```
feat(nav): add mobile menu toggle  
fix(form): stop submit on empty email  
docs(readme): add setup steps  
refactor(css): extract colour variables
```

**<type>(<scope>): <short summary in the imperative>**

types: feat · fix · docs · refactor · test · chore

Commit message quality is graded. So is committing steadily instead of one giant commit before the deadline.

# Mistakes everyone makes in week two

`"command not found: git"`

Git is not installed, or the terminal was open before installing. Close and reopen the terminal.

`"fatal: not a git repository"`

You are standing in the wrong folder. Run `pwd`, then `cd` into your project.

`"nothing to commit"`

You never staged the changes. Run `git add .` and then `git status` again.

`"Please tell me who you are"`

Set your identity once: `git config --global user.name` and `user.email`.

`node_modules pushed to GitHub`

You committed before writing `.gitignore`. Fix the ignore file and remove it from tracking.

# This week's work

## Lab 2 — Build your course repository

- 1 Install and configure VS Code, Git, and Node.js. Verify each with a version command.
- 2 Create a repository named fswd-<your-name> with folders for all sixteen weeks.
- 3 Add a README describing yourself, the course, and the repository structure.
- 4 Add a .gitignore before your first commit.
- 5 Make at least eight commits with meaningful messages, then push to GitHub.
- 6 Submit the repository link before the deadline.

## Home task

Complete the 25-command terminal drill sheet and record every command with its output in a markdown file.

## Reminder

Phase A: no AI for this lab. Type every command yourself — that is how the muscle memory forms.

NEXT WEEK

## Week 3 — HTML5: structure and meaning

### Semantics

Tags that carry meaning

### Forms

Collecting data properly

### Accessibility

Pages everyone can use

### Validation

Markup that passes the  
W3C

*Come with your environment working and your repository already pushed. We write code from minute one.*

## Questions?